

AHS VAULT

Complete Project Documentation

Zero-Knowledge Biometric Vault System

Version 0.1.20 | July 2026

Confidential - Internal Use Only

Desktop	Backend	Web App	Mobile
Tauri v2 + Rust React + TypeScript	Go 1.25 PostgreSQL 15	React 19 + TypeScript Vite 8 + Tailwind	Kotlin + Jetpack Compose Android SDK 34

Table of Contents

1. Project Overview

- 1.1 Mission & Vision
- 1.2 Core Architecture
- 1.3 System Flow
- 1.4 Tech Stack Summary

2. Desktop Application (Tauri/Rust)

- 2.1 Architecture Overview
- 2.2 Core Modules
- 2.3 Cryptographic Engine
- 2.4 Virtual Filesystem (VaultFS)
- 2.5 WebSocket Client
- 2.6 UI Components & Screens
- 2.7 Tauri Commands Reference
- 2.8 Build Configuration

3. Backend API (Go/PostgreSQL)

- 3.1 Architecture Overview
- 3.2 API Endpoints Reference
- 3.3 Database Schema
- 3.4 WebSocket Hub
- 3.5 Google Drive Storage
- 3.6 WebAuthn Verification
- 3.7 Deployment (Docker)

4. Web Authentication App (React)

- 4.1 Architecture Overview
- 4.2 Cryptographic Design
- 4.3 State Machine
- 4.4 Pages & Components
- 4.5 Hooks & Services
- 4.6 IndexedDB Schema
- 4.7 Build & Deployment

5. Mobile Authentication App (Kotlin)

- 5.1 Architecture Overview
- 5.2 Hardware-Backed Security
- 5.3 Key Components
- 5.4 Biometric Integration

5.5 Build Configuration

6. Security Design

6.1 Zero-Knowledge Principles

6.2 Identity System

6.3 Encryption Layers

6.4 Decoy PIN System

6.5 Auto-Lock & Transfer Guard

6.6 Orphan Healing & Reconciliation

7. API Reference (Complete)

7.1 Authentication

7.2 Vault Operations

7.3 Device Management

7.4 WebSocket Protocol

7.5 Web Endpoints

7.6 Error Handling

8. Deployment & Operations

8.1 Docker Compose Production

8.2 CI/CD Pipelines

8.3 Auto-Update System

8.4 Environment Variables

8.5 Monitoring & Health Checks

9. Development Guide

9.1 Prerequisites

9.2 Local Development Setup

9.3 Patch Files Reference

9.4 Code Architecture Patterns

9.5 Testing Strategy

10. Appendix

10.1 Cryptographic Constants

10.2 Message Protocol Reference

10.3 File Structure Map

1. Project Overview

1.1 Mission & Vision

AHS Vault is a **zero-knowledge biometric vault** system that provides secure cloud storage with a revolutionary architecture: the user's smartphone (or web browser) acts as a biometric remote control to unlock encrypted files on a desktop computer. The fundamental principle is that **the server never sees plaintext data** -- all encryption and decryption happens exclusively on client devices.

The system eliminates the need for traditional passwords by using biometric authentication (Face ID, Touch ID, fingerprint) as the trust anchor. The smartphone becomes the ultimate key to the desktop vault, creating a seamless yet highly secure user experience.

1.2 Core Architecture

The system consists of four main components that communicate through a central Go backend:

Component	Tech Stack	Responsibility
Desktop App	Tauri v2 (Rust + React)	Vault file management, encryption, virtual filesystem
Backend API	Go 1.25 + PostgreSQL	Relay, storage, WebSocket routing
Web App	React 19 + TypeScript	Biometric authentication, QR pairing
Mobile App	Kotlin + Jetpack Compose	Biometric authentication, QR pairing

1.3 System Flow

The complete unlock flow operates as follows:

- Step 1:** Desktop generates a 24-word BIP-39 mnemonic and derives cryptographic keys
- Step 2:** Mobile/Web scans a QR code to cryptographically pair with the desktop
- Step 3:** Desktop encrypts files with AES-256-GCM and syncs encrypted blobs to Google Drive
- Step 4:** To unlock, desktop sends a push notification through the backend to the mobile device
- Step 5:** User authenticates via biometrics (Face ID / Touch ID / fingerprint) on the mobile device
- Step 6:** Mobile decrypts the master key and sends it back to the desktop via encrypted channel (X25519 + AES-GCM)
- Step 7:** Desktop mounts a virtual filesystem (FUSE on Linux/macOS, WebDAV on Windows) that transparently decrypts files

1.4 Tech Stack Summary

Layer	Technologies
Desktop	Tauri v2, Rust, React, TypeScript, Vite 7, Tailwind CSS 4
Cryptography	Ed25519, X25519, AES-256-GCM, BIP-39, ECDSA P-256, PBKDF2
Backend	Go 1.25, Chi router, PostgreSQL 15, Gorilla WebSocket, pgx/v5
Web App	React 19, TypeScript 6, Vite 8, Tailwind CSS 4, Dexie.js, @noble/curves
Mobile	Kotlin, Jetpack Compose, Google Tink, CameraX, ML Kit, BiometricPrompt
Storage	Google Drive API (primary), MinIO (alternative)
CI/CD	GitHub Actions, Docker, Tauri Updater

2. Desktop Application (Tauri/Rust)

2.1 Architecture Overview

The desktop application is built with Tauri v2, combining a Rust backend for performance-critical operations (cryptography, filesystem management, networking) with a React/TypeScript frontend for the user interface. The application runs with **window decorations disabled** and uses a custom title bar component.

Key capabilities:

- Generate BIP-39 mnemonics and derive cryptographic keys
- Mount/unmount virtual filesystem (FUSE or WebDAV)
- Encrypt/decrypt files with AES-256-GCM (128KB blocks, parallelized)
- Sync encrypted blobs to Google Drive
- Multi-device pairing and management
- Auto-lock on inactivity with transfer guard
- Intelligent Shield email scanning
- Auto-update via GitHub releases

2.2 Core Modules

Module	Responsibility
lib.rs	Main Tauri app entry, all commands, vault mount/unmount, onboarding, pairing, auto-lock watcher, WebDAV server
crypto.rs	AES-256-GCM encrypt/decrypt (128KB blocks), BIP-39 mnemonic derivation, ECIES for mobile, stream upload/download
fs.rs	VaultFS: In-memory filesystem (FUSE), file management, encrypted shadow blobs, priority-queue sync, cloud reconciliation
network.rs	WebSocket client with auto-reconnect, handles unlock_approved, push_relay, connection_established messages
shield.rs	Intelligent Shield: IMAP email scanning for phishing detection
oauth.rs	Google OAuth2 token management and refresh
config.rs	Backend URL configuration
drive_mirror.rs	SHA-256 hash caching for skip-unchanged-files sync optimization

2.3 Cryptographic Engine

The cryptographic engine (**crypto.rs**) handles all encryption, decryption, and key derivation operations:

Algorithm	Use Case	Details
AES-256-GCM	File encryption	128KB blocks, random 12-byte nonce per block, parallelized via rayon
BIP-39	Mnemonic generation	24-word mnemonic, HD key derivation (Ed25519 + X25519)
ECIES	Mobile communication	X25519 ECDH + AES-GCM for secure key exchange with mobile
Ed25519	Request signing	API request authentication (upload, delete, pair)
SHA-256	Content hashing	Deduplication and skip-unchanged optimization

The encryption process splits files into 128KB blocks. Each block is encrypted independently with its own random 12-byte nonce, enabling parallel encryption/decryption via the rayon thread pool. A single active block per file handle provides block-level caching for performance.

2.4 Virtual Filesystem (VaultFS)

VaultFS provides a transparent decryption layer that makes encrypted cloud storage appear as a local drive. The implementation differs by platform:

Platform	Mechanism	Implementation	Notes
Linux/macOS	FUSE	fuser crate, mounted at ~/SecureVault	Full POSIX filesystem semantics
Windows	WebDAV	dav-server + tiny_http on 127.0.0.1:8081	Mapped as network drive M:\

Files are stored locally as encrypted shadow blobs. The sync worker uses a priority queue to background-sync changes to Google Drive. Cloud reconciliation handles orphan healing (index repair on mount) and garbage collection of orphaned blobs.

2.5 WebSocket Client

The WebSocket client (`network.rs`) maintains a persistent connection to the backend for real-time communication:

- Auto-reconnect with exponential backoff
- Handles messages: `unlock_approved`, `push_relay`, `connection_established`
- Desktop registration with public key and pairing nonce
- Push notification relay for multi-device unlock

2.6 UI Components & Screens

Component	Screen	Description
App.tsx	Main controller	Onboarding flow, lock screen, dashboard routing, auto-update check
Dashboard.tsx	Main dashboard	Vault explorer, device management, security center, recovery center, settings
LockScreen.tsx	Lock screen	Unlock via Phone and Offline Master Key options
VaultExplorer.tsx	File browser	Virtual filesystem file browsing with upload/download/delete
DeviceManagement.tsx	Devices	Paired mobile device management (add/remove, real-time status)
MasterKeyScreen.tsx	Mnemonic	24-word mnemonic display and restoration input
SecurityCenter.tsx	Security	Security settings and threat monitoring
RecoveryCenter.tsx	Recovery	Recovery options and backup management
AutoLockSettings.tsx	Auto-lock	Inactivity timeout configuration
TitleBar.tsx	Custom title bar	Window controls (decorations disabled)
RestorationProgress.tsx	Restoration	Progress UI for cloud vault restoration

2.7 Tauri Commands Reference

Command	Description
generate_desktop_identity	Creates BIP-39 mnemonic, derives keys, starts WebSocket, returns QR payload
mount_vault	Mount the virtual filesystem (FUSE or WebDAV)
lock_vault	Unmount the virtual filesystem and clear keys from memory
unlock_offline	Unlock using mnemonic directly (no mobile needed)
restore_vault	Full restoration from cloud using mnemonic
request_unlock_push	Sends WAKE_UP_BIOMETRIC to all paired devices
share_master_key_with_phone	Encrypts and sends master key to phone
factory_reset	Wipes all local configuration and data
check_onboarding / complete_onboarding	Onboarding state management
add_mobile_device / remove_mobile_device / get_mobile_devices	Multi-device management
save_google_tokens / is_google_connected	Google Drive OAuth integration
upload_to_vault / download_from_vault / delete_from_vault	File operations
sync_now / cleanup_orphaned_blobs	Sync and maintenance
set_auto_lock_timeout / reset_idle_timer	Inactivity auto-lock
export_vault_archive	Local backup export

2.8 Build Configuration

The desktop app is built with Tauri v2 using Vite as the bundler. Key configuration:

Setting	Value
Framework	Tauri v2
Frontend Bundler	Vite 7
CSS Framework	Tailwind CSS 4
Language	TypeScript 5.8
Rust Edition	2021
Target Version	0.1.20
Window Decorations	Disabled (custom title bar)
Auto-Update	Tauri Updater plugin + GitHub Releases

Setting	Value
Signing	TAURI_SIGNING_PRIVATE_KEY

3. Backend API (Go/PostgreSQL)

3.1 Architecture Overview

The backend is a **pure relay** -- it never sees plaintext keys or files. Built with Go 1.25, it uses the Chi router for HTTP, gorilla/websocket for real-time communication, and PostgreSQL 15 for persistence. Google Drive serves as the primary encrypted blob storage.

The backend's responsibilities are intentionally minimal: store encrypted blobs, route WebSocket messages by public key identity, manage device registrations, and proxy GitHub releases for auto-updates.

3.2 API Endpoints Reference

Method	Path	Handler	Description
GET	/health	HealthCheck	Health check endpoint
GET	/api/update	HandleUpdate	Proxies GitHub releases for Tauri auto-updater
POST	/api/vault/upload	UploadVault	Upload encrypted blob (Ed25519 signed)
GET	/api/vault/download/{blob_id}	DownloadVault	Download encrypted blob
POST	/api/vault/pair	PairVault	Native mobile pairing (signature verified)
POST	/api/vault/push	RelayPush	Relay encrypted blob to target device via WS
POST	/api/vault/shield/log	LogThreat	Log threat events from Shield
POST	/api/vault/register	RegisterDevice	Register/update device info
GET	/api/vault/devices	GetDevices	List devices by public key
DELETE	/api/vault/devices	DeleteDevice	Remove device (bulk or single)
GET	/api/vault/activity	GetActivity	Get activity logs
GET	/api/vault/stats	GetStats	Dashboard stats (score, files, storage, threats)
GET	/api/vault/index	GetRootIndex	Get root blob index pointer
POST	/api/vault/index	SetRootIndex	Set root blob index pointer
POST	/api/vault/delete	DeleteVault	Delete encrypted blobs
GET	/api/ws/connect	WsConnect	Native WebSocket upgrade
POST	/api/web/pair	WebPairVault	Web app pairing

Method	Path	Handler	Description
POST	/api/web/push	WebRelayPush	Web unlock relay (WebAuthn verified)
POST	/api/web/register-webauthn	WebRegisterWebAuthn	Register WebAuthn credential
GET	/api/web/ws/connect	WebWsConnect	Web app WebSocket upgrade

3.3 Database Schema

Devices Table

```
CREATE TABLE devices (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  public_key TEXT UNIQUE NOT NULL,  
  name TEXT NOT NULL,  
  os TEXT NOT NULL,  
  status TEXT NOT NULL DEFAULT 'secure',  
  last_active TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  last_root_blob_id TEXT  
);
```

Activity Logs Table

```
CREATE TABLE activity_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  device_public_key TEXT NOT NULL,  
  event_type TEXT NOT NULL,  
  title TEXT NOT NULL,  
  description TEXT NOT NULL,  
  risk_level TEXT NOT NULL DEFAULT 'low',  
  timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

Blobs Table

```
CREATE TABLE blobs (  
  blob_id TEXT PRIMARY KEY,  
  owner_public_key TEXT NOT NULL,  
  size_bytes BIGINT NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

3.4 WebSocket Hub

The WebSocket hub ([internal/websocket/hub.go](#)) manages all real-time connections and message routing:

- Identity binding: Desktop sends {type: 'desktop_register', public_key, pairing_nonce}
- Mobile/Web sends {type: 'mobile_register', public_key}
- Messages routed by public key identity
- Pairing nonces are single-use and expire after 5 minutes
- Server sends: unlock_approved, push_relay, WAKE_UP_BIOMETRIC, connection_established

3.5 Google Drive Storage

The Google Drive integration ([internal/storage/gdrive.go](#)) handles encrypted blob persistence:

- Upload encrypted blobs with MIME type application/octet-stream
- Download by blob ID with streaming support

- Delete with confirmation
- Folder management per user (identified by public key)
- Storage statistics (total size, file count)

3.6 WebAuthn Verification

The WebAuthn module ([internal/auth/webauthn.go](#)) verifies WebAuthn assertions from the web app:

- ECDSA P-256 public key verification
- SHA-256 challenge hashing
- Signature verification using crypto/ecdsa
- Credential ID to public key lookup

3.7 Deployment (Docker)

The backend is containerized with a multi-stage Docker build:

```
# Multi-stage build
FROM golang:1.25-alpine AS builder
WORKDIR /app
COPY . .
RUN go build -o server ./cmd/api

FROM alpine:latest
COPY --from=builder /app/server /server
EXPOSE 8080
CMD ["/server"]
```

Service	Image	Port	Notes
backend	Go API server	8080	Depends on db
db	PostgreSQL 15 Alpine	5432	Volume: postgres_prod_data

4. Web Authentication App (React)

4.1 Architecture Overview

The web authentication app is a React 19 single-page application that serves as one of the two biometric authentication endpoints (the other being the Android app). It runs entirely in the browser with local-first data storage via IndexedDB (Dexie.js).

Component	Technology
Framework	React 19 + TypeScript 6
Build Tool	Vite 8
Styling	Tailwind CSS 4
Cryptography	@noble/curves (X25519), @noble/hashes, WebCrypto API
WebAuthn	@simplewebauthn/browser (L3)
Storage	Dexie.js (IndexedDB)
Routing	react-router-dom v7
QR Code	html5-qrcode scanner
BIP-39	@scure/bip39

4.2 Cryptographic Design

Purpose	Algorithm	Details
Identity	ECDSA P-256	Generated on first visit, stored in IndexedDB
Transport	X25519 DH + AES-256-GCM	Ephemeral key per unlock session
PIN	PBKDF2-HMAC-SHA-256	310,000 iterations + 16-byte random salt
Biometrics	WebAuthn L3	Platform authenticator (Face ID/Touch ID), ES256
Decoy PIN	Separate PBKDF2 hash	Opens fake vault under duress

4.3 State Machine

The web app follows a strict state machine for onboarding and authentication:

```
loading -> onboarding (3-step carousel)
onboarding -> main (Unpaired)
main (Unpaired) + QR scan -> pairing
pairing -> security-setup (PIN -> Decoy PIN -> Biometric)
security-setup -> main (Locked)
main (Locked) + Biometric/PIN -> main (Unlocked)
```

4.4 Pages & Components

Component	Screen	Description
Dashboard.tsx	Main dashboard view	Security hero card, quick actions, live status grid
Settings.tsx	Settings page	Biometric enrollment/removal, 24-word recovery, vault config, theme, factory reset
Shield.tsx	Security monitoring	Threat detection display and security status
Activity.tsx	Activity log	Device activity and event history
BiometricPrompt.tsx	Biometric overlay	Prompts for fingerprint/face authentication
PinPad.tsx	PIN entry	5-digit PIN pad with decoy support
Scanner.tsx	QR scanner	Camera-based QR code scanner for pairing
FloatingNavBar.tsx	Navigation	Bottom navigation bar (Vault, Shield, Activity, Settings, Devices)
SecurityHeroCard.tsx	Status card	Security status display
QuickActionButtons.tsx	Actions	Quick action buttons
LiveStatusGrid.tsx	Monitoring	Real-time monitoring grid

4.5 Hooks & Services

Module	Responsibility
useWebAuthn.ts	WebAuthn registration/authentication via @simplewebauthn/browser (Local-First pattern)
useWebSocket.ts	WebSocket connection to backend with auto-reconnect, handles push_relay messages
api.ts	HTTP API calls: pairDevice, sendUnlockApproval, getVaultStats
crypto.ts	All cryptographic operations: ECDSA P-256, X25519, AES-GCM, PBKDF2, signature conversion
db.ts	Dexie.js database: identity keys, PIN hash/salt, pairing data, biometric credentials, master key

4.6 IndexedDB Schema (Dexie.js)

Database: AHSVaultAuth

Stores:

- identity: ECDSA P-256 keypair + X25519 keypair
- pairing: desktop public key, backend URL, session data
- config: PIN hash, PIN salt, decoy PIN hash, decoy salt
- biometric: WebAuthn credential ID, public key
- vault: encrypted master key, mnemonic (encrypted)

```
- settings: theme, auto-lock preferences
```

5. Mobile Authentication App (Kotlin)

5.1 Architecture Overview

The mobile authentication app is a native Android application built with Kotlin and Jetpack Compose. It provides hardware-backed biometric authentication and serves as the primary key to the desktop vault.

Component	Technology
Language	Kotlin
UI Framework	Jetpack Compose
Target SDK	26-34 (compile SDK 34)
Build System	Gradle with Kotlin DSL
Cryptography	Google Tink (X25519, AES-GCM), Android Keystore (P-256 ECDSA)
Biometrics	androidx.biometric (BiometricPrompt API)
Camera	CameraX 1.3 + ML Kit Barcode Scanning
Networking	OkHttp 4.12, Gson
Version	1.0 (versionCode 1)

5.2 Hardware-Backed Security

The Android app leverages hardware security features for maximum protection:

- **TEE/Strongbox:** P-256 ECDSA keys generated in Trusted Execution Environment
- **Invalidated by Biometric Enrollment:** Keys auto-deleted if new fingerprint added
- **Encrypted SharedPreferences:** Sensitive data stored with hardware-backed encryption
- **Android Keystore:** Private keys never leave secure hardware

5.3 Key Components

Component	Responsibility
MainActivity.kt	Central coordinator for all security operations
CryptoManager	Hardware-backed key generation (P-256 in TEE/Strongbox), X25519 via Tink, AES-GCM
SecureStorageManager	Encrypted SharedPreferences/DataStore for key persistence
WebSocketService.kt	Foreground service for persistent WebSocket connection
BiometricPrompt	Biometric authentication integration

Component	Responsibility
CameraX + ML Kit	QR code scanning during pairing

5.4 Biometric Integration

The mobile app uses Android's BiometricPrompt API for secure biometric authentication. The integration includes:

- Fingerprint, face, and iris authentication support
- Hardware-backed key binding (keys unusable without biometric)
- Automatic key invalidation on biometric enrollment changes
- Fallback to device credential (PIN/pattern) when available

6. Security Design

6.1 Zero-Knowledge Principles

AHS Vault implements true zero-knowledge architecture with these guarantees:

- **No plaintext on server:** All encryption/decryption happens on client devices only
- **No server-side keys:** Cryptographic keys are derived from BIP-39 mnemonics on client devices
- **Encrypted blobs:** Server stores only opaque encrypted data, cannot read content
- **No password transmission:** Biometric authentication replaces traditional passwords
- **Client-side key derivation:** Master keys never leave the device

6.2 Identity System

Component	Identity	Notes
Desktop	Ed25519 signing key + X25519 encryption secret	Derived from BIP-39 mnemonic
Mobile/Web	ECDSA P-256 identity key + X25519 transport key	Hardware-backed where possible
Pairing	QR code exchange of public keys + backend URL + nonce	One-time pairing nonce, 5min expiry

6.3 Encryption Layers

The system employs multiple encryption layers for defense in depth:

Layer	Purpose	Algorithm	Details
Layer 1	File Encryption	AES-256-GCM	128KB blocks, random nonce per block
Layer 2	Transport Encryption	X25519 ECDH + AES-GCM	Ephemeral session keys for key exchange
Layer 3	Identity Signing	Ed25519 / ECDSA P-256	API request authentication
Layer 4	PIN Protection	PBKDF2-HMAC-SHA-256	310K iterations, 16-byte salt
Layer 5	Hardware Binding	Android Keystore / WebAuthn	Keys bound to biometric

6.4 Decoy PIN System

AHS Vault includes a plausible deniability feature via decoy PINs. When a user is under duress, they can enter a decoy PIN that opens a fake vault with harmless content. The decoy PIN is stored as a separate PBKDF2 hash, making it computationally indistinguishable from the real PIN.

- Separate PBKDF2 hash for decoy PIN
- Decoy vault contains innocuous files

-
- No indication to attacker which PIN is real
 - Configurable during security setup

6.5 Auto-Lock & Transfer Guard

The auto-lock system clears all cryptographic keys from RAM after a configurable inactivity period (default 300 seconds). The Transfer Guard prevents auto-lock during active file transfers using an RAll pattern:

- Inactivity timer resets on user interaction
- Configurable timeout (default 300 seconds)
- Transfer Guard: RAll lock prevents auto-lock during transfers
- Immediate lock on manual trigger
- Memory-only viewing: files decrypted only in RAM

6.6 Orphan Healing & Reconciliation

The system includes automatic repair mechanisms for maintaining data integrity:

- **Orphan healing:** Index repair on mount for corrupted or partial states
- **Cloud reconciliation:** Background purging of garbage/orphaned blobs
- **SHA-256 caching:** Skip unchanged files during sync for efficiency
- **Priority queue sync:** Background worker with configurable priority

7. API Reference (Complete)

7.1 Authentication

All API requests from the desktop are signed with Ed25519. The signature covers the request body and is included in the X-Signature header. The backend verifies the signature against the registered public key.

7.2 Vault Operations

Upload Encrypted Blob

```
POST /api/vault/upload
Headers: X-Public-Key, X-Signature, Content-Type: application/octet-stream
Body: <encrypted binary data>
```

```
Response: { "blob_id": "...", "size": 12345 }
```

Download Encrypted Blob

```
GET /api/vault/download/{blob_id}
Headers: X-Public-Key, X-Signature
```

```
Response: <encrypted binary data>
```

Delete Encrypted Blobs

```
POST /api/vault/delete
Headers: X-Public-Key, X-Signature
Body: { "blob_ids": ["id1", "id2"] }
```

7.3 Device Management

Register Device

```
POST /api/vault/register
Body: { "public_key": "...", "name": "...", "os": "..." }
```

List Devices

```
GET /api/vault/devices?public_key=...

Response: [{ "id": "...", "name": "...", "status": "secure", ... }]
```

Delete Device

```
DELETE /api/vault/devices
Body: { "public_key": "...", "device_ids": ["..."] }
```

7.4 WebSocket Protocol

WebSocket connections use JSON messages. The protocol supports two registration types:

```
// Desktop Registration
{
  "type": "desktop_register",
  "public_key": "ed25519-public-key-hex",
  "pairing_nonce": "random-nonce"
}

// Mobile/Web Registration
{
  "type": "mobile_register",
  "public_key": "ecdsa-public-key-hex"
}

// Server Messages
{ "type": "unlock_approved", ... }
{ "type": "push_relay", "data": "<encrypted-blob>" }
{ "type": "WAKE_UP_BIOMETRIC" }
{ "type": "connection_established" }
```

7.5 Web Endpoints

Method	Path	Description
POST	/api/web/pair	Web app pairing with desktop
POST	/api/web/push	Web unlock relay (WebAuthn verified)
POST	/api/web/register-webauthn	Register WebAuthn credential
GET	/api/web/ws/connect	Web app WebSocket upgrade

7.6 Error Handling

All API errors return JSON with a consistent format:

```
{
  "error": "<error message>",
  "code": "<error_code>"
}
```

Common error codes:

- 400 - Bad Request (invalid signature, missing fields)
- 401 - Unauthorized (invalid/missing public key)
- 404 - Not Found (blob_id does not exist)
- 500 - Internal Server Error

8. Deployment & Operations

8.1 Docker Compose Production

```
version: '3.8'

services:
  backend:
    build: ./vault-backend-go
    ports:
      - "8080:8080"
    depends_on:
      - db
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/vault
      - GOOGLE_DRIVE_CREDENTIALS=/secrets/gdrive.json

  db:
    image: postgres:15-alpine
    volumes:
      - postgres_prod_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=vault
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass

volumes:
  postgres_prod_data:
```

8.2 CI/CD Pipelines

Windows Desktop Build (build-windows.yml)

- Trigger: Push to main or tag matching v*
- Environment: windows-latest
- Setup: Node.js 22, Rust stable
- Build: npm install && npm run tauri build
- Sign: TAURI_SIGNING_PRIVATE_KEY
- Artifacts: .exe, .msi, .zip, .sig
- Release: GitHub Release on tag push

Mobile Build (build-mobile.yml)

- Trigger: Push to main
- Environment: ubuntu-latest
- Setup: JDK 17 (Temurin)
- Build: ./gradlew assembleDebug
- Artifact: app-debug.apk

8.3 Auto-Update System

The auto-update system works as follows:

1. Desktop app queries backend `/api/update` on startup
2. Backend proxies GitHub Releases API
3. Extracts `.nsis.zip` and `.sig` URLs from latest release
4. Tauri updater plugin downloads and verifies signature
5. Silent install via NSIS installer
6. App relaunches automatically via `tauri-plugin-process`

8.4 Environment Variables

Variable	Description	Default/Example
DATABASE_URL	PostgreSQL connection string	postgres://user:pass@localhost:5432/vault
GOOGLE_DRIVE_CREDENTIALS	Path to Google Drive service account JSON	/secrets/gdrive.json
PORT	HTTP server port	8080
TAURI_SIGNING_PRIVATE_KEY	Tauri auto-update signing key	Required for desktop builds
TAURI_SIGNING_PRIVATE_KEY_PASSWORD	Password for signing key	Optional

8.5 Monitoring & Health Checks

The backend exposes a health check endpoint at `GET /health` that returns the service status. Docker Compose can use this for container health checks.

- `GET /health` returns `{ "status": "ok" }`
- PostgreSQL connection pool health via `pgx`
- WebSocket hub connection count
- Google Drive API quota monitoring

9. Development Guide

9.1 Prerequisites

Component	Requirements
Desktop	Node.js 22+, Rust stable, Tauri CLI
Backend	Go 1.25+, PostgreSQL 15+, Google Drive credentials
Web App	Node.js 22+, npm
Mobile	JDK 17 (Temurin), Android SDK 34, Gradle
Docker	Docker Engine 20+, Docker Compose v2

9.2 Local Development Setup

Backend

```
cd vault-backend-go
go mod download
go run ./cmd/api
```

Desktop

```
cd vault-desktop-tauri
npm install
npm run tauri dev
```

Web App

```
cd vault-web-auth
npm install
npm run dev
```

Mobile

```
cd vault-mobile-auth
./gradlew assembleDebug
```

9.3 Patch Files Reference

The project includes several patch files used during development to apply incremental changes:

File	Purpose
patch_config.py	Adds MobileDevice struct and mobile_devices field to OnboardingConfig
patch_lib.py	Updates push logic for multi-device, adds device management Tauri commands
patch_app.py	Fixes onboarding step logic after pairing

File	Purpose
patch_device.py	Updates DeviceManagement.tsx for local device management
patch_ts.py	TypeScript fixes (imports, types, unused parameters)
apply_process_plugin.js	Adds tauri-plugin-process for app relaunch capability

9.4 Code Architecture Patterns

Pattern	Description
Zero-Knowledge Relay	Backend never sees plaintext; all crypto on clients
Virtual Filesystem Abstraction	FUSE (Linux/macOS) and WebDAV (Windows) behind common interface
Priority Queue Sync	Background sync with configurable priority and transfer guard
RAII Transfer Guard	Prevents auto-lock during active file transfers
Hardware-Backed Identity	Keys bound to biometric hardware (Android Keystore / WebAuthn)
Dual Authentication Paths	Biometric primary, PIN fallback, offline mnemonic recovery
Plausible Deniability	Decoy PIN system for duress scenarios
Orphan Healing	Automatic index repair on mount for corrupted states

9.5 Testing Strategy

The testing strategy covers multiple levels:

- **Unit Tests:** Cryptographic operations, key derivation, encryption/decryption
- **Integration Tests:** API endpoints, WebSocket protocol, database operations
- **E2E Tests:** Full pairing flow, unlock flow, file operations
- **Security Tests:** Signature verification, key isolation, memory clearing
- **Platform Tests:** FUSE/WebDAV mounting, auto-update, CI/CD builds

10. Appendix

10.1 Cryptographic Constants

Constant	Value
AES Block Size	128KB (131,072 bytes)
AES Nonce Size	12 bytes (96 bits)
AES Key Size	32 bytes (256 bits)
PBKDF2 Iterations	310,000
PBKDF2 Salt Size	16 bytes
BIP-39 Wordlist	2048 words (English)
Mnemonic Length	24 words
Ed25519 Key Size	32 bytes
X25519 Key Size	32 bytes
ECDSA P-256 Key Size	32 bytes (private), 64 bytes (public)
SHA-256 Digest	32 bytes
Pairing Nonce Expiry	300 seconds (5 minutes)
Default Auto-Lock	300 seconds (5 minutes)

10.2 Message Protocol Reference

All WebSocket messages use JSON format with a 'type' field for routing:

Message Type	Direction	Description
desktop_register	Desktop -> Server	Register desktop identity with public key and pairing nonce
mobile_register	Mobile -> Server	Register mobile identity with public key
unlock_approved	Server -> Desktop	Mobile approved unlock, contains encrypted master key
push_relay	Server -> Desktop	Relay encrypted blob from mobile
WAKE_UP_BIOMETRIC	Server -> Mobile	Wake up mobile for biometric authentication
connection_established	Server -> Both	Confirm WebSocket connection is active
pairing_request	Mobile -> Server -> Desktop	Initiate pairing with public key exchange

Message Type	Direction	Description
pairing_complete	Server -> Both	Pairing handshake completed

10.3 File Structure Map

```

ahs-app/
├── vault-desktop-tauri/ # Desktop app (Tauri v2)
│   ├── src-tauri/
│   │   ├── src/
│   │   │   ├── lib.rs # Main Tauri commands
│   │   │   ├── main.rs # Entry point
│   │   │   ├── crypto.rs # AES-256-GCM, BIP-39, ECIES
│   │   │   ├── fs.rs # VaultFS (FUSE/WebDAV)
│   │   │   ├── network.rs # WebSocket client
│   │   │   ├── shield.rs # Email phishing scanner
│   │   │   ├── oauth.rs # Google OAuth2
│   │   │   ├── config.rs # Configuration
│   │   │   └── drive_mirror.rs # SHA-256 hash cache
│   │   ├── Cargo.toml
│   │   └── tauri.conf.json
│   └── src/
│       ├── App.tsx # Main React app
│       ├── screens/
│       │   ├── Dashboard.tsx
│       │   └── LockScreen.tsx
│       ├── components/
│       │   ├── VaultExplorer.tsx
│       │   ├── DeviceManagement.tsx
│       │   ├── MasterKeyScreen.tsx
│       │   ├── SecurityCenter.tsx
│       │   ├── RecoveryCenter.tsx
│       │   ├── AutoLockSettings.tsx
│       │   ├── TitleBar.tsx
│       │   └── RestorationProgress.tsx
│       └── package.json
└── vault-backend-go/ # Backend API (Go)
    ├── cmd/api/main.go # Entry point
    ├── internal/
    │   ├── api/
    │   │   ├── router.go # Route definitions
    │   │   ├── handlers.go # API handlers
    │   │   └── web_handlers.go # Web-specific handlers
    │   ├── db/
    │   │   ├── db.go # PostgreSQL layer
    │   │   └── json_db.go # JSON fallback
    │   └── auth/
    │       ├── webauthn.go # WebAuthn verification
    │       └── websocket/
    │           ├── hub.go # WebSocket hub
    │           └── storage/
    │               ├── gdrive.go # Google Drive storage
    │               └── minio.go # MinIO storage
    ├── Dockerfile
    └── go.mod

```

-
- vault-web-auth/ # Web auth app (React)
 - ■■■■ src/
 - ■ ■■■■ App.tsx # Main app controller
 - ■ ■■■■ main.tsx # React root
 - ■ ■■■■ lib/
 - ■ ■ ■■■■ crypto.ts # Cryptographic operations
 - ■ ■ ■■■■ db.ts # Dexie.js IndexedDB
 - ■ ■■■■ hooks/
 - ■ ■ ■■■■ useWebAuthn.ts # WebAuthn hook
 - ■ ■ ■■■■ useWebSocket.ts # WebSocket hook
 - ■ ■■■■ services/
 - ■ ■ ■■■■ api.ts # HTTP API calls
 - ■ ■■■■ components/
 - ■ ■ ■■■■ BiometricPrompt.tsx
 - ■ ■ ■■■■ PinPad.tsx
 - ■ ■ ■■■■ Scanner.tsx
 - ■ ■ ■■■■ FloatingNavBar.tsx
 - ■ ■■■■ screens/
 - ■ ■ ■■■■ Dashboard.tsx
 - ■ ■ ■■■■ Settings.tsx
 - ■ ■ ■■■■ Shield.tsx
 - ■ ■ ■■■■ Activity.tsx
 - ■■■■ package.json
 -
 - vault-mobile-auth/ # Mobile auth app (Kotlin)
 - ■■■■ app/
 - ■ ■■■■ build.gradle.kts
 - ■ ■■■■ src/main/
 - ■ ■■■■ AndroidManifest.xml
 - ■ ■■■■ java/com/vault/auth/
 - ■ ■■■■ MainActivity.kt
 - ■ ■■■■ CryptoManager.kt
 - ■ ■■■■ SecureStorageManager.kt
 - ■ ■■■■ WebSocketService.kt
 - ■■■■ build.gradle.kts
 -
 - .github/workflows/
 - ■■■■ build-windows.yml # Desktop CI/CD
 - ■■■■ build-mobile.yml # Mobile CI/CD
 -
 - docker-compose.prod.yml # Production Docker
 - README.md
 - PROJECT.md
 - docs/
 - AHS_Vault_Complete_Documentation.pdf

End of Document

AHS Vault Complete Project Documentation
Version 0.1.20 | Generated July 23, 2026
Confidential - Internal Use Only